

Familiar Guest — Solution Design

Reference design: features & value, architecture, data quality, operations, personas

A solution-level design for the Familiar Guest platform: a direct-booking and guest-management product for US/Canadian owners renting homes in Mexico. It maps features to business value, defines the recommended component architecture, the data-quality and operational-monitoring model with automated resolution, and the user personas and their interactions.

1 • Features & Business Value

Feature	What it does	Business value
Branded booking link	Private, mobile, no guest account; owner's property name	Keeps the guest relationship; avoids Airbnb's ~15.5% fee
Guest CRM	Private directory: history, notes, trust tiers	The retention moat — repeat bookings, higher LTV, lock-in
Escrow (delayed payout)	Funds held until check-in	Core consumer trust; makes cross-border direct booking safe
Multi-currency + cross-border payouts	Price in USD/CAD/MXN; pay out to US/CA/MX bank	Key differentiator for US/CA owners with MX property
Bilingual + AI translation	EN/ES booking & messaging	Expands guest base to MX nationals; removes owner effort
iCal calendar sync	Inbound + outbound feeds	Prevents double-bookings; no per-OTA development
Rental agreement (DocuSeal)	Auto-generated, e-signed, stored	Legal protection; professional credibility
Damage deposit + Protected Booking	Hold + up to \$1M coverage (Truvi)	Owner risk mitigation; high-margin add-on revenue
Guest screening (Truvi)	ID + fraud check	Trust for unknown guests; add-on revenue
Verified Owner (Stripe KYC)	Identity verification gate	Guest trust + compliance; condition of payout
AI listing description	Owner content → polished listing	Fast, low-effort onboarding
Automated messaging (Resend)	Confirmation → checkout sequence	Consistent guest experience; less owner work
Digital house manual	Private link, Baja templates	Fewer guest questions; better stays/reviews
Caretaker role + digital check-in	Scoped local access; self-arrival	Enables absentee ownership
Income reporting	Summary + consolidated (Pro)	Owner value at tax time; Pro upsell
Season re-invite + one-click rebook	Timed Nov-Apr nudges	Recurring revenue from the existing guest base

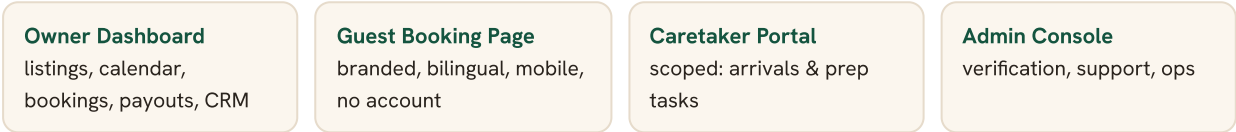
Long-stay rates + installments	Weekly/monthly + split pay	Fits snowbird behavior; closes large bookings
Rental Income & Tax Payment Accounting	Track rental income; calculate/collect/remit lodging taxes; withhold & remit Mexican host taxes; documents for tax reporting across units	Core differentiator — solves the hardest part of cross-border renting; no competitor offers it; drives retention & trust (handling + reporting, not advice)
FX spread (recommended)	Modest transparent currency margin	GMV-linked revenue — the hedge against OpEx outrunning revenue

2 • Architecture — Recommended Components

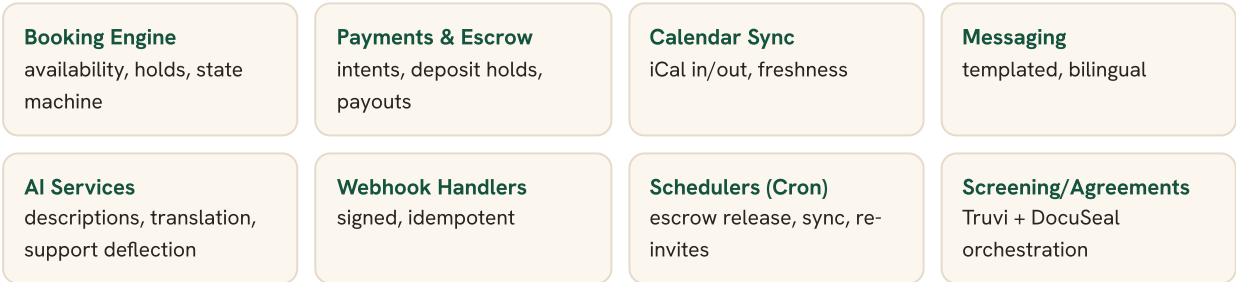
USERS



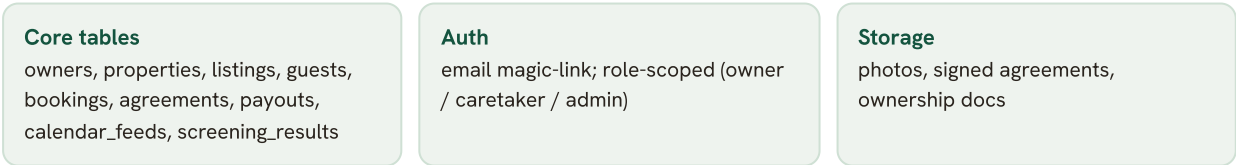
EXPERIENCE LAYER — NEXT.JS (APP ROUTER) ON VERCEL



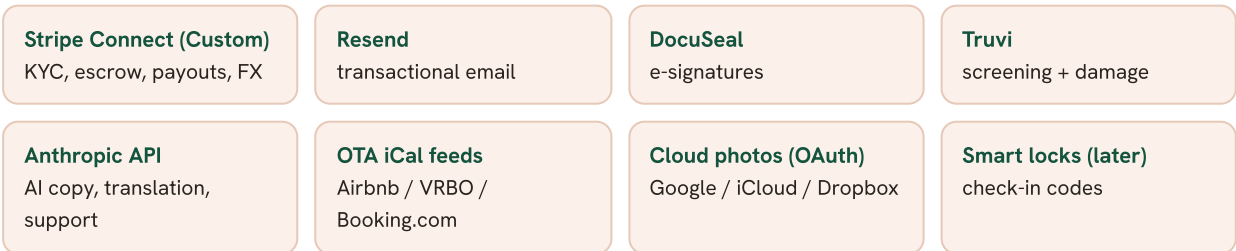
APPLICATION & ORCHESTRATION — SERVERLESS FUNCTIONS + VERCEL CRON



DATA — SUPABASE (POSTGRES + AUTH + STORAGE), ROW-LEVEL SECURITY



EXTERNAL INTEGRATIONS



CROSS-CUTTING — OBSERVABILITY & OPS

Sentry — errors/traces

Logs & metrics —
structured, per-event

Alerting — thresholds →
on-call

Auto-remediation —
retry, resync, dunning

Design principles: serverless-first (no servers to manage), webhooks always signature-verified and idempotent, money actions never auto-executed outside safe idempotent bounds, PII minimized (Stripe-hosted KYC — no SSNs stored), and every external call wrapped with retries + a dead-letter path.

3 • Data Quality

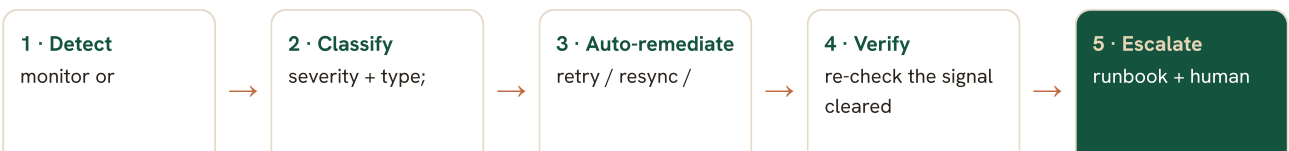
Domain	Controls	Why it matters
Financial integrity	Webhook signature verification; idempotency keys; nightly Stripe↔DB reconciliation; amounts stored in minor units + currency code; captured FX rate per transaction	No double charges, lost payments, or mismatched payouts
Booking integrity	Explicit state machine; DB constraints; availability locks preventing overlapping dates	Eliminates double-bookings and invalid states
Calendar data	iCal parse validation; per-feed "last synced" timestamp; stale-feed (>45 min) detection	Availability stays accurate across platforms
Property location	GPS coordinates (lat/long) required for Mexican listings — addresses are unreliable; validate range	Accurate map display and Google Maps directions in check-in messages
Identity / PII	Stripe-hosted KYC (no SSN/ID stored); RLS on every table; encryption at rest; least-privilege roles	Compliance and security; limits breach blast radius
CRM records	Schema validation; email/phone normalization; duplicate detection/merge	A clean, trustworthy guest list (the moat)
Documents	Signed-agreement integrity (immutable store + checksum); retention policy	Defensible legal record per booking
Cross-border money	Currency code on every amount; FX rate snapshot; payout-destination validation	Accurate multi-currency accounting and payouts

4 • Operational Monitoring & Automated Resolution

Monitored signals → automated response

Signal	Detection	Automated resolution	Escalate when
App errors / latency	Sentry + Vercel	Auto-rollback on bad deploy; surface error	Error spike persists
Stripe webhooks	Delivery / signature failures	Idempotent replay from queue	Dead-letter queue grows
Payment / payout failure	Failed intent / payout	Smart retries + dunning; notify owner	Retries exhausted
Escrow-release job	Cron heartbeat	Re-run idempotent job	Missed window
Calendar freshness	last-synced age	Auto re-fetch feed	Feed fails repeatedly → flag
Email delivery	Resend bounce/fail	Re-send; suppress hard bounces	Domain reputation drop
Agreement signing	DocuSeal status	Auto reminder to guest	Stuck past threshold
Screening	Truvi timeout/error	Retry; fall back to manual review	Repeated failure
Support load	Ticket volume/topic	AI deflection + auto-resolve known issues	Novel/complex issue

Automated resolution process



webhook fires a
signal

auto-eligible &
idempotent-safe?

dunning / re-send /
re-run job

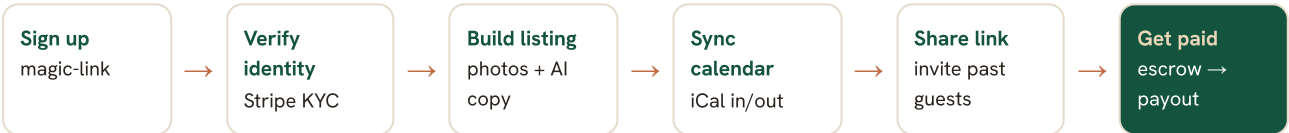
(founder/support) if
unresolved

Safety rule: financial actions are auto-retried only within idempotent, bounded limits; anything that could move or release funds incorrectly stops and escalates to a human. Every escalation feeds a post-incident step that updates the runbook and, where possible, adds a new monitor or auto-remediation so the same issue self-heals next time.

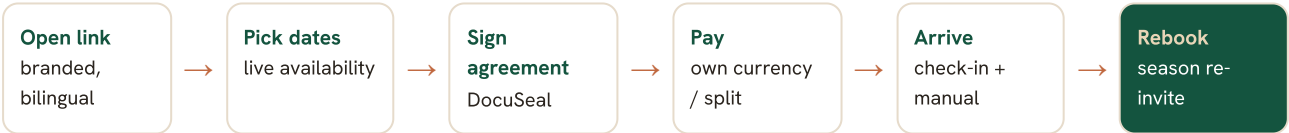
5 • User Personas & Expected Interactions

Persona	Who they are	Primary goals	Key interactions
Owner	US/Canadian; owns 1-10 units in Mexico ; rents on Airbnb, has repeat guests, mostly absentee	Take repeat guests direct across all their units, keep earnings, get paid cross-border, minimal effort	Onboard & verify, build listings (often several), sync calendars, share links, manage multi-unit bookings/payouts, work the guest CRM
Returning Guest	From US, Canada, or Mexico; knows/trusts the owner; may be a snowbird	Book a trusted home easily, pay safely in own currency & language	Open link, pick dates, sign agreement, pay (or split), get check-in info & house manual
Caretaker	Local (Mexico) cleaner/property manager	Know who's arriving and prepare the home	Scoped portal: upcoming arrivals, prep tasks, check-in details (no payments / no full guest list)
Founder / Admin	Platform operator (you)	Keep the platform trustworthy and running; support owners	Review property-ownership (Gate 2) docs, handle escalations, watch monitoring dashboards

Owner journey



Guest journey



Interaction notes

- **Owner ↔ Guest** messaging is bilingual and AI-translated both ways; the owner never needs Spanish, the guest never needs English.
- **Caretaker** sees only operational data for confirmed stays — never payment or guest-contact detail beyond what's needed to host.
- **Admin** touches a booking only on exception (verification review or an escalated incident); the happy path is fully automated.
- **Multi-unit owners** manage several listings under one account; consolidated income reporting (Pro), scoped caretaker access, and a single shared guest CRM span all their units.